



Automate the Cropping of Photos from Scanned Album Pages

This month, we switch to a new track. Let's explore how Imagemagick kills the drudgery in extracting photographs from scanned pages by automating the process.

How does one easily share memories stored in photo albums going back generations? Simple Scan is really handy. It scans a page, but extracting photos from the scanned image is a very tedious task. So, this month, let's automate this tedious task to reduce the drudgery. This is clearly an example of xkcd automation (<http://xkcd.com/1319/>)! However, it also falls into the category of spending time on 'what doesn't seem like work!' (<http://www.paulgraham.com/work.html>). There is a wonderful script called 'Multicrop' (<http://www.fmwconcepts.com/imagemagick/multicrop/>), which uses Imagemagick tools to crop and straighten images.

Extending the concept behind Multicrop

The basic sequence of how the Multicrop script works is as follows:

- 'Fuzzy' replaces the background colour by none and the rest by red
- Actual images will be islands of red surrounded by 'none'
- It extracts each red island and binds or surrounds it in a rectangle
- It uses the rectangle as a mask on the original image, before extracting the photo

A fuzz factor is used to select the background colour.

If the value is too high, part of the background of the photo may be lost and the photo may be split into multiple parts. If the value is too low, photos may not be extracted. However, even if a part of the photo is treated as background, as long as the enclosing rectangle is the size of the original photo, you don't have to worry.

The script worked very well with multiple loose photos scanned at one time, as long as there was some gap between the photos and the boundaries.

My problem was that the photos could not be removed from the album without damage. Besides, the background was not uniform, but comprised multiple colours. Hence, in the above sequence, I decided to replace the first step by three others:

- Select a set of colours from the border
- Replace each colour by 'none'

- Replace what is left of the image with red

This helped reduce the drudgery though definitely did not save any time!

Imagemagick steps

The following steps have been adapted from the Multicrop script referred to above, though I converted the steps into a Python script using `os.system`, `subprocess.call` and `subprocess.check_output` methods. For more details about the conversion options, see <http://www.imagemagick.org/script/command-line-options.php>. Sample values have been used where needed to simplify the examples.

Convert the image file (Figure 1) into Imagemagick's internal `mpc` format, and use the latter for the intermediate steps for efficient processing:

```
convert image.jpg +repage out.mpc
```

For each background colour - `bgcolor` as a (r,g,b) tuple, rename `out.mpc` and `out.cache` to `in.mpc` and `in.cache` and floodfill; 'none' should replace the background colour. A 1x1 pixel border of the background colour is added to ensure floodfilling is from all sides of the image and it is then shaved off.

```
rename out in out.*
```

```
convert in.mpc -fuzz 6% -fill none \
    -bordercolor srgb+str(bgcolor) -border 1x1 \
    -draw matte 0,0 floodfill -shave 1x1 out.mpc
```

The next step is to remove the background and replace the remaining image with red. You will get an image similar to what's shown in Figure 2.

```
convert out.mpc -fuzz 6% -fill red +opaque none \
    -background black -alpha background TMP2.mpc
```

You now need to find a cluster of red pixels. Since your photo will not be very small, rather than searching pixel by pixel, you can speed up the process by a factor of 100 by searching every 10th pixel in each row and column.